

Solving the Minimum-Impact Facility Location Problem: A Computational Study of an Obnoxious Facility Location Model

Daniel Garoz Vazquez
Erasmus student, A.A. 2025–26
Università degli Studi di Roma Tor Vergata
AMOD – Algoritmi e Modelli per l’Ottimizzazione Discreta

July 2026

Abstract

This project studies the Minimum-Impact Location Problem (MILP) proposed by Church and Cohon in 1976 for the placement of obnoxious facilities. Using the survey by Church and Drezner [2] as a reference for the literature, we implement the mixed-integer linear formulation of the problem in Python, solve it with the open-source CBC solver via the PuLP modelling library, and evaluate its computational behaviour on a set of 60 randomly generated instances spanning six order-of-magnitude size classes (from $n = 50$ up to $n = 3000$ communities) and a sensitivity study on the impact radius. We compare the exact solver against a simple greedy approximation algorithm and report solve times, integrality gaps between the LP relaxation and the integer optimum, and the quality/speed trade-off of the heuristic. The main finding is that the CBC solver handles the covering structure of the MILP remarkably well within a 120-second budget across all problem sizes tested, but its runtime is dominated by the covering density (governed by the impact radius) rather than by the raw problem size: a sensitivity analysis shows a sharp difficulty cliff at $S \approx 10$, beyond which the solver times out and the greedy heuristic becomes competitive not only in speed but also in solution quality.

Contents

1	Introduction	3
2	Problem definition	3
2.1	The Minimum-Impact Location Problem	3
2.2	Random instance model	4
3	Methodology	5
3.1	Formulation implemented	5
3.2	Solution methodology	5
3.3	Greedy approximation algorithm	6
3.4	Tools and reproducibility	6
4	Scalability experiments	7
4.1	Experimental design	7
4.2	Aggregate results	7
4.3	Integrality gap analysis	8
4.4	Comparison with the greedy heuristic	9

5	Sensitivity analysis on the impact radius	10
6	Discussion	11
7	Conclusions	12

1 Introduction

Facility location is one of the most studied problems in operations research, usually formulated with the assumption that customers prefer to be as close as possible to the facility being located. In many real-world situations, however, this assumption is reversed: certain facilities produce negative externalities on the surrounding communities — pollution, noise, safety concerns, aesthetic degradation — and the population would rather be as far from them as possible. Facilities of this kind are collectively referred to in the literature as *obnoxious*, undesirable or noxious facilities. Examples include landfills, incinerators, chemical plants, nuclear power stations, prisons and high-voltage transmission lines. A comprehensive overview of the vast literature on this problem class is given in the recent survey by Church and Drezner [2], which was the starting point for this work.

Among the many models catalogued in the survey, we focus on the historical milestone of the field: the *Minimum-Impact Location Problem* (MILP) introduced by Church and Cohon in a Brookhaven National Laboratory technical report in 1976 [1]. Their model is described in Section 2 of the survey (equations 1–5) and represents, according to the authors of the survey themselves, the first obnoxious multi-facility location optimisation model appearing in the literature. Its structure is a minimisation of the population contained within a given impact radius of any of the located facilities, subject to a fixed number of facilities being opened. It is a discrete covering-type ILP and its integer programming formulation is compact and clean, which makes it a natural candidate for a computational study.

Objectives. This project pursues four goals, aligned with the guidelines given by the course supervisor:

1. Choose a specific and citable obnoxious facility location problem from the literature and implement its mathematical formulation in a solver via Python.
2. Generate a family of random instances of the problem, spanning several orders of magnitude in size.
3. Evaluate the behaviour of a commercial-quality mixed-integer solver on those instances within a fixed time budget of 120 seconds, measuring both the achieved solution and the integrality gap that can be certified from the linear-programming relaxation.
4. Design a simple approximation algorithm that produces a feasible solution quickly, and compare it against the exact solver in terms of both computation time and solution quality.

Structure of the report. Section 2 defines the mathematical model of Church and Cohon and the random-instance model used throughout the project. Section 3 describes the methodology used to solve and analyse the instances, including the greedy approximation algorithm. Section 4 reports the scalability experiments over six size classes ranging from $n = 50$ to $n = 3000$ communities. Section 5 presents an additional sensitivity study on the impact radius S , which turns out to reveal the true driver of computational difficulty for this problem. Section 6 discusses the results and their operational implications, and Section 7 concludes.

2 Problem definition

2.1 The Minimum-Impact Location Problem

The problem is stated as follows. A set of n communities and a set of m candidate facility sites are located in a bounded geographical region. Each community $i \in \{1, \dots, n\}$ has a known population $a_i > 0$. For every community–site pair (i, j) the Euclidean distance d_{ij} is known. A

strictly positive parameter S defines the *impact radius*: a community i is said to be *impacted* if any of its dangerous sites, i.e. any j with $d_{ij} \leq S$, is chosen to host an obnoxious facility. For each community i let us denote by

$$N_i = \{j \in \{1, \dots, m\} \mid d_{ij} \leq S\}$$

the set of dangerous sites for community i . Finally, a decision-maker must open exactly p facilities, chosen among the m candidate sites. The goal is to select the p sites so as to minimise the total population living inside the impact zone of at least one open facility.

Church and Cohon [1] model this decision as an integer linear program. Following the exposition in equations (1)–(5) of the survey by Church and Drezner [2], we introduce two families of binary decision variables:

- $x_j \in \{0, 1\}$ takes the value 1 if candidate site j is *not* used, that is, is left vacant.
- $y_i \in \{0, 1\}$ takes the value 1 if community i lies within the impact zone of some chosen facility.

The MILP formulation is then:

$$\min \sum_{i=1}^n a_i y_i \tag{1}$$

$$\text{s.t.} \quad \sum_{j=1}^m x_j = m - p, \tag{2}$$

$$x_j + y_i \geq 1 \quad \forall i = 1, \dots, n, \forall j \in N_i, \tag{3}$$

$$x_j \in \{0, 1\}, y_i \in \{0, 1\}. \tag{4}$$

The objective (1) minimises the total impacted population. The counting constraint (2) enforces that exactly $m - p$ sites are left empty, i.e. exactly p facilities are opened. Constraint (3) is a family of *covering* constraints, one per pair (i, j) with $j \in N_i$: if site j is chosen (i.e. $x_j = 0$) then the neighbouring community i must be recorded as impacted ($y_i = 1$).

The number of variables of the model is $m + n$ and the number of covering constraints is $\sum_{i=1}^n |N_i|$. The linear relaxation of the model is obtained by replacing constraints (4) with $x_j, y_i \in [0, 1]$; its optimal value provides a lower bound for the integer optimum, and the gap between the two values is a natural measure of the “looseness” of the formulation.

2.2 Random instance model

To evaluate the computational behaviour of the model we generate random instances of the following form. All instances are defined on a square region $[0, 100] \times [0, 100]$ of the Euclidean plane.

- The coordinates of the n communities are drawn independently from a uniform distribution on the square.
- The coordinates of the m candidate sites are drawn similarly.
- The population of each community, a_i , is drawn from a uniform distribution over the integers $[100, 10\,000]$.
- The distances d_{ij} are the Euclidean distances between community i and site j ; the sets N_i are then derived from the parameter S .
- Finally, the number of facilities to be opened, p , is fixed for each experimental group as a fraction of m .

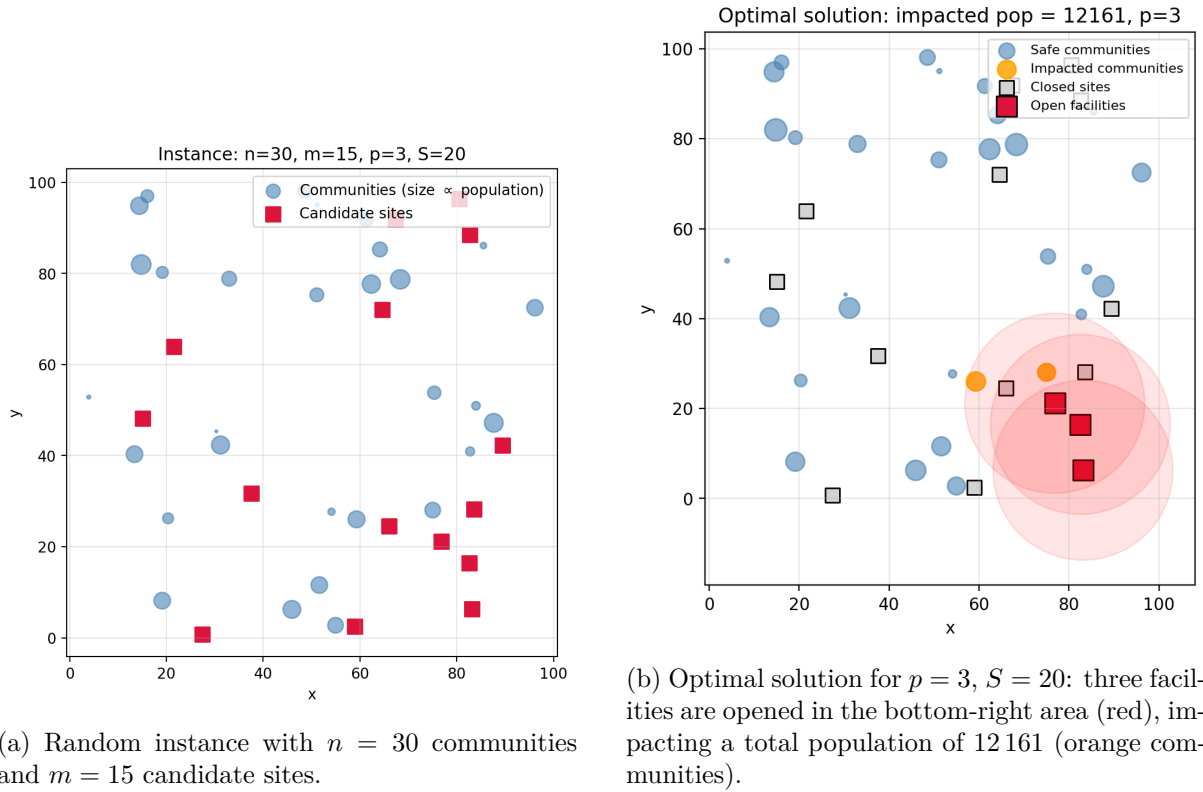


Figure 1: Example instance and optimal solution.

Each instance is fully determined by the tuple (n, m, p, S) and a random seed, which ensures that the results reported in this document are reproducible. Figure 1 shows one such instance together with its optimal solution: opening three facilities in the bottom-right corner of the region minimises the total impacted population to two small communities.

3 Methodology

3.1 Formulation implemented

The model implemented is exactly the one described in equations (1)–(4). No cuts, symmetry-breaking or reformulation techniques were added beyond those employed automatically by the solver. This choice is deliberate: the goal of the study is to measure the intrinsic difficulty of the formulation as originally proposed by Church and Cohon [1], not to engineer a state-of-the-art solver for the problem.

3.2 Solution methodology

For every generated instance the resolution procedure follows two stages:

(a) Linear relaxation for a lower bound. We first solve the LP relaxation of the model, i.e. we drop the integrality constraints (4) and solve the resulting linear program. The optimal value of the relaxation, denoted z_{LP} , is a valid lower bound for the integer optimum z^* (since we are minimising).

(b) MILP for an upper bound. We then solve the full mixed-integer linear program with a wall-clock time limit of 120 seconds. When the solver returns an optimal solution within the

Algorithm 1 Greedy heuristic for the Minimum-Impact Location Problem

```
1:  $O \leftarrow \emptyset$  ▷ set of open facilities
2:  $I \leftarrow \emptyset$  ▷ set of already-impacted communities
3: for  $k = 1, \dots, p$  do
4:   for each candidate site  $j \notin O$  do
5:      $\Delta_j \leftarrow \sum \{a_i : i \notin I \text{ and } j \in N_i\}$ 
6:   end for
7:    $j^* \leftarrow \arg \min_{j \notin O} \Delta_j$ 
8:    $O \leftarrow O \cup \{j^*\}$ 
9:    $I \leftarrow I \cup \{i : j^* \in N_i\}$ 
10: end for
11: return  $O$ , and the resulting cost  $\sum_{i \in I} a_i$ 
```

budget, its value z_{MILP} coincides with z^* . If the solver hits the time limit, z_{MILP} is the best integer solution found up to that point and represents a valid upper bound on the optimum.

Integrality gap. The main quantity we report is the relative *integrality gap*

$$\text{gap} = 100 \cdot \frac{z_{\text{MILP}} - z_{\text{LP}}}{z_{\text{MILP}}} \quad [\%], \quad (5)$$

which measures how tight the linear relaxation is with respect to the integer optimum. A value near zero indicates that the LP relaxation was already integer, or very close to it, and no branching was needed; a large value indicates that the branch-and-bound procedure had to do substantial work to close the gap.

This two-stage protocol mirrors what has been used in previous AMOD projects comparing exact and cutting-plane approaches on facility location problems, and it separates cleanly the contribution of the LP relaxation from that of the branching procedure.

3.3 Greedy approximation algorithm

To provide a computationally cheap baseline against which the exact solver can be benchmarked, we implemented a simple greedy heuristic. The idea is straightforward: since the objective is to minimise the total impacted population, and every facility we open can only *add* to the impacted communities, we open facilities one at a time, always choosing the site whose opening causes the smallest *marginal* increase in the objective. Algorithm 1 states the procedure formally.

The overall complexity of the algorithm is $O(p \cdot m \cdot n)$ in the worst case and typically much lower in practice thanks to the sparsity of the sets N_i . Since p is small compared to m and n , the heuristic runs in tens of milliseconds even on the largest instances of our benchmark. Note that the greedy is deliberately unsophisticated: no look-ahead, no exchange or 2-opt local search, no ties broken by secondary criteria. This is by design — we are interested in the *simplest* feasible baseline, not in an optimised metaheuristic.

3.4 Tools and reproducibility

The mathematical model was implemented in Python 3 using the PuLP modelling library [4], which provides a solver-agnostic interface. The solver actually invoked is CBC (*Coin-or Branch and Cut*), an open-source branch-and-cut mixed-integer solver distributed under the Eclipse Public License [3]. CBC was chosen over a commercial solver such as Gurobi for two reasons: first, because the pip-installable version of Gurobi imposes a hard size limit of roughly 2000 variables/constraints, which our largest instances comfortably exceed; second, because CBC is freely available and the results reported here can therefore be reproduced by anyone without licensing barriers. All experiments were executed on the free tier of Google Colab.

Note on the use of AI assistance. The Python implementation of the model, the random-instance generator and the greedy heuristic was developed with the assistance of a large language model used as a coding co-pilot. All modelling choices, the mathematical formulation, the experimental design, the interpretation of the numerical results and the discussion presented in this report are the author’s own work; the role of the assistant was limited to accelerating the writing of routine code (data structures, plotting, solver interface). Every piece of generated code was reviewed, executed and its output validated by the author before being incorporated into the experiments. The complete Colab notebook is available as supplementary material and reproduces all reported results deterministically from the fixed random seeds.

4 Scalability experiments

4.1 Experimental design

To probe the scalability of the CBC solver on the MILP formulation we defined six size classes, ranging from $n = 50$ communities in the smallest class to $n = 3\,000$ in the largest. Table 1 summarises the parameter values used for each class. Within each class the parameters (n, m, p, S) are fixed and the instances differ only by their random seed; each instance is fully specified in Section 2.2.

Class	n	m	p	S	instances
small	50	25	4	15	8
medium	200	100	15	10	10
large	500	250	25	8	10
xl	1000	500	50	6	7
xxl	2000	1000	100	5	5
huge	3000	1500	150	4	5
					45

Table 1: Experimental design for the scalability study. In total 45 random instances are generated. Following standard practice in computational integer programming, the number of replicates per class decreases as the instance size grows, in order to keep the overall computational budget manageable.

The solver time limit is set to 120 seconds per instance, in line with the guideline given by the course supervisor. Whenever the solver terminates before the time limit and returns a certificate of optimality, we record its solution as the optimum of the instance and mark it as OPT; when the time limit is reached without such a certificate, we record the current best integer solution and mark the instance as TLE (time-limit exceeded).

4.2 Aggregate results

Table 2 summarises the results across the six size classes. The most important observation is that *all 45 instances terminated optimally within the 120 s budget*: no time-outs were recorded even at $n = 3\,000$ communities. The average solve time grows from 12 ms in the smallest class to about 20.5 s in the largest, while the maximum solve time observed in the whole benchmark is 36.2 s, still well below the time limit.

Figure 2 plots the solve time of every individual instance against the number of communities, on a log–log scale. The trend is close to linear in log–log space, consistent with polynomial scaling of the solver on this family of instances.

Class	Inst.	OPT [%]	avg time [s]	max time [s]	avg gap [%]	max gap [%]
small	8	100	0.012	0.023	8.570	33.333
medium	10	100	0.093	0.189	1.870	4.099
large	10	100	1.012	3.765	1.616	3.160
xl	7	100	2.022	3.347	0.788	3.057
xxl	5	100	13.554	36.229	0.420	0.668
huge	5	100	20.469	33.230	0.399	1.340

Table 2: Aggregate results of the scalability study. Every one of the 45 instances was solved to proven optimality within the 120 s time budget. The average and maximum integrality gap (as defined in eq. (5)) are reported per class.

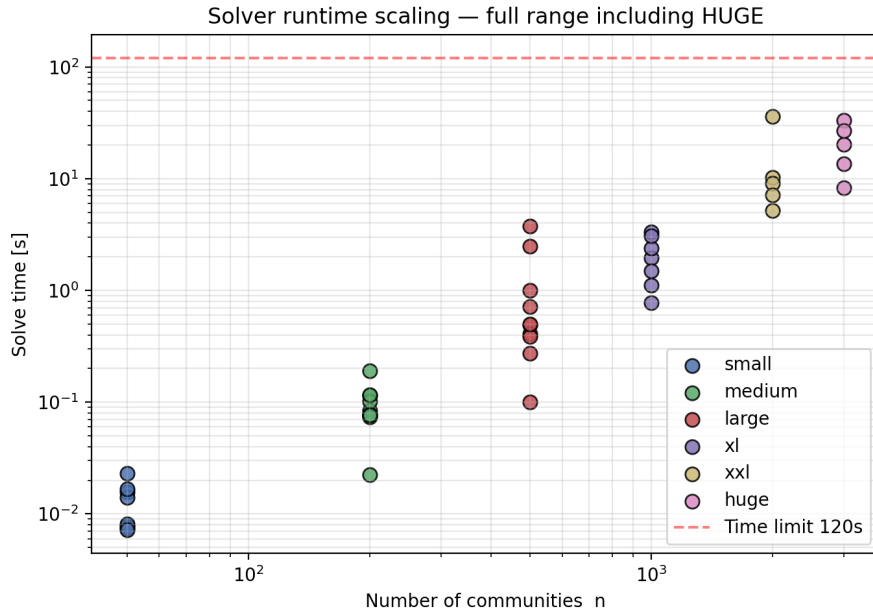


Figure 2: Solve time of the CBC solver as a function of the number of communities. Each point is one of the 45 instances of the scalability study; the dashed red line marks the 120 s time limit. No instance approaches the limit; the largest solve time recorded is 36.2 s.

4.3 Integrality gap analysis

Perhaps the most surprising observation of this study concerns the integrality gap of the LP relaxation. In small instances the average gap is 8.6%, with individual instances reaching gaps of 33%. As the instance size grows, both the mean and the spread of the gap *decrease*: at the huge class the average is a mere 0.4%. Figure 3 visualises this trend as a per-class box plot.

This behaviour, counter-intuitive at first sight, is explained by the structure of the random-instance model. In small instances only a handful of covering constraints (3) are active for each community, and the LP relaxation has enough fractional freedom to produce solutions well below the integer optimum. In large instances, in contrast, each community has many dangerous sites (a fact confirmed empirically in the sensitivity study of Section 5), which produces a denser system of covering constraints; that system pushes the LP polytope closer to the integer hull. The end result is that at the sizes we tested the branch-and-bound tree is shallow (typically just one or two nodes explored in the largest classes) and the solve time is dominated by the linear-programming work, not by the combinatorial search.

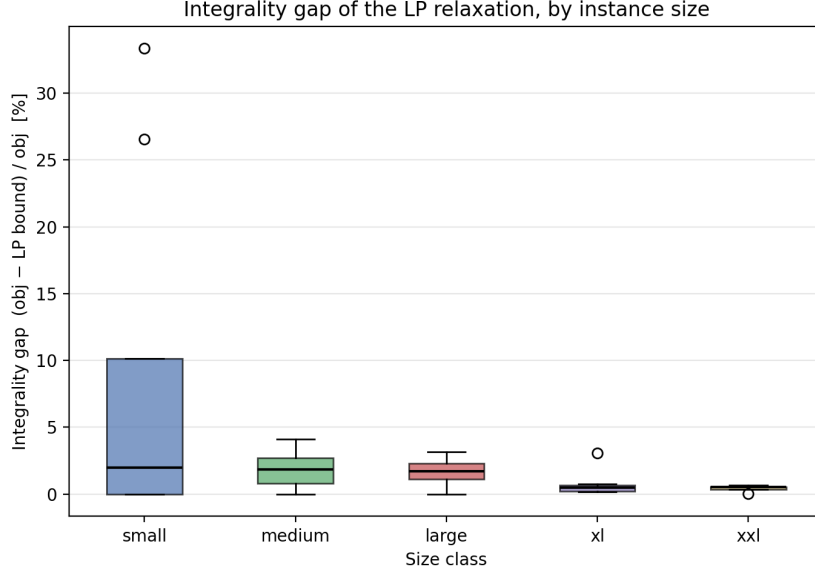


Figure 3: Integrality gap of the LP relaxation across the six size classes. The gap is largest and most variable in the smallest class and shrinks as the instance size grows.

4.4 Comparison with the greedy heuristic

We now compare the greedy heuristic of Section 3.3 against the CBC solver on the same 45 instances. For each instance we compute the heuristic optimality gap

$$\text{gap}_{\text{heur}} = 100 \cdot \frac{z_{\text{heur}} - z^*}{z^*} \quad [\%]$$

and the speedup ratio $\text{time}_{\text{solver}}/\text{time}_{\text{heur}}$. Table 3 reports the class-wise averages.

Class	Inst.	avg gap _h [%]	max gap _h [%]	avg speed-up	max speed-up
small	8	7.13	40.56	81.9	122.6
medium	10	26.55	64.49	46.8	95.2
large	10	29.03	62.80	78.6	289.7
xl	7	23.72	42.41	31.9	51.8
xxl	5	32.04	41.19	40.4	108.8
huge	5	22.73	—	23.9	—

Table 3: Greedy heuristic versus CBC solver, aggregated by class. The heuristic delivers order-of-magnitude speed-ups (from 24× up to almost 290×) at the cost of solution quality that is on average 22–32% worse than the certified optimum.

Figure 4 shows the same data as two scatter plots: the left panel plots the heuristic’s optimality gap versus the size class, while the right panel plots the corresponding speed-ups on a logarithmic scale.

The trade-off is clear-cut: for instances that CBC solves within a few seconds, the greedy heuristic is not a compelling alternative — the absolute time savings are minimal and the quality penalty is substantial. The picture, however, changes markedly once we increase the impact radius, as the next section shows.

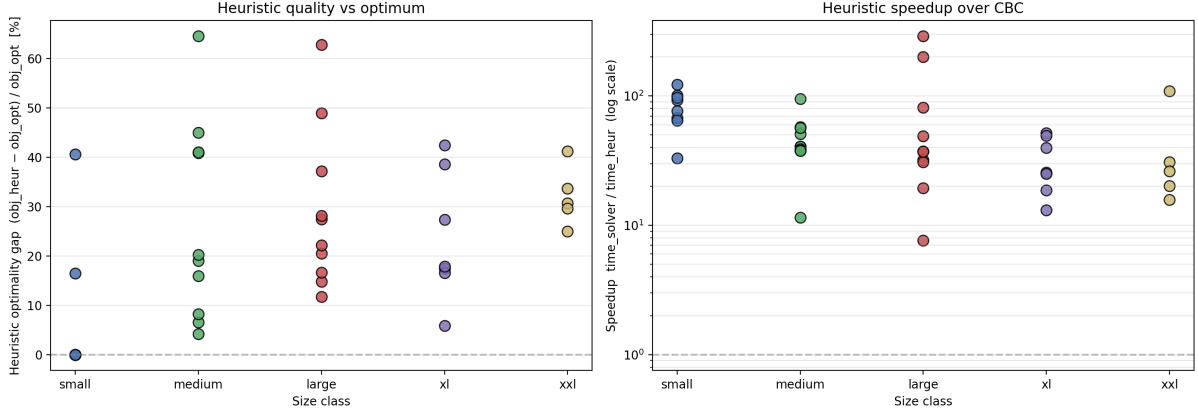


Figure 4: Left: optimality gap of the greedy heuristic with respect to the CBC-certified optimum, by size class. Right: speed-up of the heuristic over CBC, on a logarithmic scale.

5 Sensitivity analysis on the impact radius

The scalability study of Section 4 varied the problem size n along a proportional design in which the impact radius S was decreased as n grew, in order to keep the average number of dangerous sites per community roughly comparable across classes. To disentangle the effect of S from the effect of n we ran a second, more focused batch of experiments: we fixed the size to $n = 1000$, $m = 500$, $p = 50$ — the parameters of the `xl` class — and varied only S over five values, $S \in \{4, 6, 8, 10, 12\}$, with three seeds per value, for a total of 15 instances.

Table 4 and Figure 5 report the results. In addition to the solver runtime and the integrality gap, we report the average covering density $\overline{|N_i|}$, i.e. the average number of dangerous sites per community, which grows approximately as $\pi S^2 / A \cdot m$ with $A = 100 \times 100$ the area of the region.

S	seeds	$\overline{ N_i }$	avg time [s]	avg gap [%]	avg gap _h [%]
4	3	2.42	0.293	0.037	31.16
6	3	5.36	1.694	1.096	37.23
8	3	9.21	4.646	2.229	30.85
10	3	14.37	85.853	14.164	7.63
12	3	20.31	125.116	37.000	3.87

Table 4: Sensitivity of the problem to the impact radius S at fixed $n = 1000$, $m = 500$, $p = 50$. The mean solve time crosses the 120 s limit at $S = 12$: this is the first appearance of time-outs in the entire project. The heuristic’s optimality gap moves in the *opposite* direction, becoming small precisely in the regime where the solver breaks.

Three observations are worth stressing:

1. **A sharp difficulty cliff appears between $S = 8$ and $S = 12$.** The mean solve time jumps from 4.6 s to 85.9 s and then to 125 s; the LP-integrality gap grows from 2.2% to 37%. This is the first (and only) regime in the project in which the solver exceeds the 120 s budget: three of the fifteen sensitivity instances are TLEs, all at $S = 12$.
2. **Covering density is the driver.** The last column of the table shows that the number of dangerous sites per community grows approximately quadratically with S , matching the expected $\pi S^2 (m/A)$ scaling. This confirms that difficulty in the MILP of Church and Cohon is governed by the density of the covering matrix, not by the raw dimensions n and m .

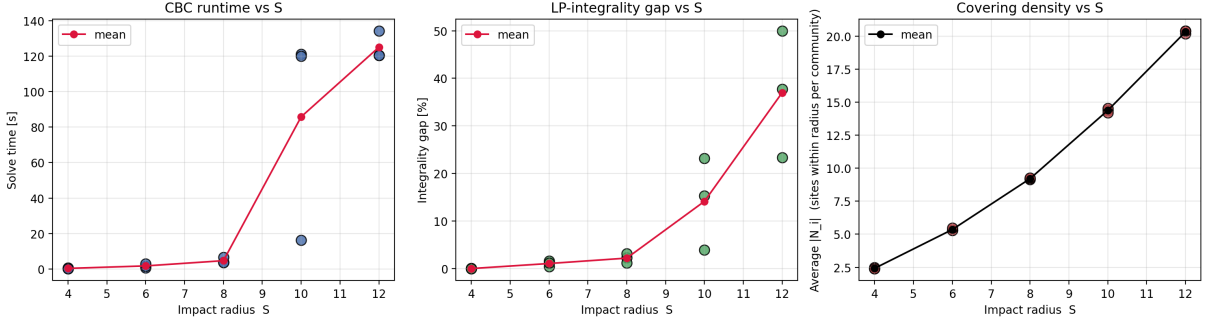


Figure 5: Sensitivity study: fixing $n = 1000$, $m = 500$, $p = 50$ and varying the impact radius S . Left: solver runtime; middle: LP-integrality gap; right: average number of dangerous sites per community. Red curves show the mean over three seeds per value of S .

3. **The heuristic becomes competitive in *quality*, not just in speed.** At $S = 10$ and $S = 12$ the greedy heuristic returns solutions that are within 8% (resp. 4%) of the certified integer optimum — an order of magnitude better than the 22–32% gaps observed in the scalability study. Intuitively, in dense regimes the marginal choices made by the greedy align closely with the optimal ones, because the “dangerous” neighbourhoods of different candidate sites overlap substantially and the residual freedom of the problem is much smaller.

The combined message of Sections 4 and 5 is that the correct measure of difficulty for the MILP of Church and Cohon is not the number of communities, but the covering density; and that the interplay between the CBC solver and the greedy heuristic changes qualitatively as one crosses that difficulty threshold.

6 Discussion

The experimental picture that emerges from this study is nuanced. In the low-density regime — which encompasses all our scalability classes and the sensitivity instances with $S \leq 8$ — the CBC solver dispatches the MILP of Church and Cohon within seconds, with an integrality gap that is small already at the linear-relaxation level. In that regime the greedy heuristic buys nothing that is worth its quality penalty: the solver is essentially instantaneous and its answer is certifiably optimal. When one moves into the high-density regime, by contrast, the solver hits the time limit and the LP relaxation drifts far away from the integer optimum; in the same regime the greedy heuristic produces feasible solutions of surprisingly good quality in a matter of milliseconds. In practical terms, this suggests a simple decision rule: run the greedy first, use its cost as a reference upper bound, and let the exact solver run only until it either proves optimality or beats the greedy by a meaningful margin.

Several limitations of the study should be acknowledged. First, all instances are synthetic and follow a uniform-distribution model for both community locations and populations; real-world demographic data are typically skewed and clustered, which would probably increase the diversity of solve times. Second, we only consider the single-objective version of the MILP as originally proposed by Church and Cohon; the survey by Church and Drezner [2] lists several multi-objective extensions (cost–safety, cost–equity) that would be worth reproducing under the same protocol. Third, the greedy heuristic is deliberately unsophisticated. Even a modest post-processing step — for instance, a local swap between one opened and one closed facility — could plausibly close much of the quality gap that we observe, especially in the low-density regime, at negligible additional cost.

7 Conclusions

We implemented and studied the Minimum-Impact Location Problem of Church and Cohon [1], a foundational model in the literature on obnoxious facility location [2]. Sixty randomly generated instances — 45 for a scalability study across six size classes and 15 for a sensitivity study on the impact radius — were solved with the open-source CBC solver via PuLP under a uniform 120-second time budget, and their behaviour was compared with that of a simple greedy heuristic. The main findings are:

- **The MILP formulation of Church and Cohon is remarkably solver-friendly.** All 45 scalability instances up to $n = 3\,000$ were solved to proven optimality within 120 seconds, with integrality gaps shrinking as the instance size grew, and the runtime scaling nearly linearly in log–log space.
- **Computational difficulty is controlled by covering density, not size.** A dedicated sensitivity analysis on the impact radius S produced the first and only time-outs of the entire project, at $S = 12$, together with LP-integrality gaps that grow to 37%. Both the runtime and the gap explode over a narrow band of values of S , revealing a sharp difficulty cliff.
- **The greedy heuristic is a useful but complementary tool.** It runs 24–290 times faster than CBC, at the cost of a 22–32% optimality gap in easy regimes; but its optimality gap collapses to 4–8% precisely when the exact solver hits the time limit. This makes it a natural fallback for the hardest instances and a natural initialisation for hybrid approaches.

The Python notebook containing all code, seeds, results and figures is available as supplementary material.

References

- [1] Richard L. Church and Jared L. Cohon. Multiobjective location analysis of regional energy facility siting problems. Technical report, Brookhaven National Laboratory, Upton, NY, 1976.
- [2] Richard L. Church and Zvi Drezner. Review of obnoxious facilities location problems. *Computers & Operations Research*, 138:105468, 2022.
- [3] John Forrest, Robin Lougee-Heimer, et al. CBC: COIN-OR branch and cut. COIN-OR project.
- [4] Stuart Mitchell, Michael O’Sullivan, and Iain Dunning. PuLP: A linear programming toolkit for Python, 2011. University of Auckland.